Sets, relations, functions, graphs, and trees solutions

B-E01 Set logic and counting

Derivation and check

Since $B \cup C = \{1, 2, 3, 4, 5, 8\}$, the difference is $A \setminus (B \cup C) = \{0, 6\}$. For every x,

$$\begin{aligned} x \in A \setminus (B \cup C) &\iff x \in A \land x \notin B \land x \notin C \\ &\iff x \in (A \setminus B) \cap (A \setminus C). \end{aligned}$$

This proves both inclusions. The set sizes are $|A| = 5$, $|B| = 4$, and $|C| = 3$. The intersections are

Intersection	Elements	Size
$A \cap B$	$\{2, 4\}$	2
$A \cap C$	$\{2, 8\}$	2
$B \cap C$	$\{2\}$	1
$A \cap B \cap C$	$\{2\}$	1

Inclusion-exclusion gives $5 + 4 + 3 - 2 - 2 - 1 + 1 = 8$, matching the union $\{0, 1, 2, 3, 4, 5, 6, 8\}$. Complements depend on U; these differences depend on the specified sets.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

B-E02 Equivalence classes and partitions

Derivation and check

Reflexivity follows from $a - a = 0$. If $a - b = 4q$, then $b - a = 4(-q)$, proving symmetry. If $a - b = 4q$ and $b - c = 4r$, then $a - c = 4(q + r)$, proving transitivity.

The classes are $\{-3, 1, 5\}$, $\{-2, 2\}$, $\{-1, 3\}$, and $\{0, 4\}$. All ordered pairs within each class belong to R, so

$$|R| = 3^2 + 2^2 + 2^2 + 2^2 = 21.$$

This count includes nine reflexive pairs.

For the general proof, suppose $c \in [a] \cap [b]$. Then aRc and bRc, so symmetry and transitivity give aRb. If $x \in [a]$, symmetry gives bRa and transitivity gives bRx. Hence $[a] \subseteq [b]$. Exchange a, b to get the reverse inclusion. Intersecting classes are equal, and removing duplicate classes yields a partition.

Sets, relations, functions, graphs, and trees solutions (continued)

B-E03 Counterexamples among relation properties

Derivation and check

1. The empty relation is symmetric and transitive because the antecedents of the required implications never hold. It is not reflexive on the nonempty set A, since it lacks (a, a).
2. The relation $R = \{(a, a), (b, b), (c, c), (a, b)\}$ is reflexive. Every composable pair already has the required resulting pair in R, so it is transitive. It is not symmetric because (b, a) is absent.
3. The equality relation $\{(a, a), (b, b), (c, c)\}$ is both symmetric and antisymmetric.

The relation $\{(a, b), (b, a), (b, c)\}$ satisfies neither property. The distinct vertices a, b are related in both directions, violating antisymmetry; the missing pair (c, b) violates symmetry. The two properties can both hold or both fail.

B-E04 Poset covers and extrema

Derivation and check

The covers are $(1, 2)$, $(1, 3)$, $(2, 6)$, $(3, 6)$, and $(3, 9)$. The pairs $(1, 6)$ and $(1, 9)$ have intermediate elements, so they are not covers.

The only minimal element is 1, which is also the minimum. The maximal elements are $6, 9$. They are incomparable, so there is no maximum. If M, N were both maxima, then $M \leq N$ and $N \leq M$, and antisymmetry would give $M = N$.

After removing 1, both 2 and 3 are minimal, and no minimum remains. The maximal elements remain $6, 9$, and there is still no maximum. Hasse edges omit both transitive and reflexive pairs.

B-E05 Codomain and inverse

Derivation and check

Equality of two f-values lets us cancel the factor 2, so f is injective. Its range is the odd integers, and it misses the even integers, so it is not onto $\mathbb{Z}$. Its inverse function is $(y + 1)/2$ only on the odd integers.

Every g-value y has preimages $2y$ and $2y + 1$. Thus g is onto $\mathbb{Z}$, has range $\mathbb{Z}$, and is not injective. Its inverse relation assigns two values to every y, so it is not a function.

Sets, relations, functions, graphs, and trees solutions (continued)

The set preimage is $f^{-1}(\{-1, 1\}) = \{0, 1\}$. Set preimages are defined without surjectivity. Restricting the codomain of f to the odd integers makes it bijective. Relation inversion swaps ordered pairs; function inversion additionally requires existence and uniqueness.

B-E06 Directed degrees and SCCs

Derivation and check

Vertex	Indegree	Outdegree
a	1	1
b	1	2
c	2	1
d	1	1
e	1	1

Both degree totals are $6 = |E|$. The SCCs are $\{a, b\}$, $\{c, d\}$, and $\{e\}$. Ignoring direction gives components $\{a, b, c, d\}$ and $\{e\}$.

The induced edges on $\{b, c, d\}$ are $b \to c$, $c \to d$, and $d \to c$. The edge $b \to a$ is excluded because a lies outside the selected vertex set.

The directed loop contributes one indegree and one outdegree. A simple undirected edge requires distinct endpoints, so the loop $e \to e$ must be removed. The vertex e remains as an isolated vertex.

B-E07 Equal degrees do not prove isomorphism

Derivation and check

Both graphs have $n = 6, m = 6$, with six vertices of degree 2. The cycle C_6 is connected, whereas two disjoint triangles have two components. An isomorphism maps walks vertex by vertex in both directions, so it preserves reachability and components. Thus the graphs are not isomorphic.

The general condition for a vertex bijection f is

$$(u, v) \in E \iff (f(u), f(v)) \in E'.$$

Sets, relations, functions, graphs, and trees solutions (continued)

With a finite vertex bijection and equal edge counts, the induced map on edges is injective. If every source edge maps to a target edge, the m distinct images exhaust the m target edges. The reverse direction then follows. Without equal edge counts, one-way preservation can embed a sparse graph in a denser one without giving an isomorphism.

B-E08 Six conditions and boundaries

Derivation and check

First prove that a tree has $n - 1$ edges. At $n = 1$, it has zero edges. For $n \geq 2$, an endpoint of a longest simple path has no extra neighbor. An outside neighbor would extend the path, while another path neighbor would create a cycle. The endpoint is therefore a leaf. Removing it and its edge preserves connectivity and acyclicity. By induction, the remaining tree has $n - 2$ edges; adding back the removed edge gives $n - 1$.

Conversely, start at one vertex of a connected graph. At each step, connectivity supplies an edge from the visited set to an unvisited vertex. Add that edge and its new vertex. No cycle forms. After $n - 1$ steps, the selected edges form a spanning tree. If the graph itself has exactly $n - 1$ edges, no extra edges remain, so it is that tree.

A triangle plus an isolated vertex has $n = 4, m = 3$ but is disconnected. At $n = 0$, universal path conditions can be vacuously true, while $m = n - 1$ would assert $0 = -1$, which is false. The equivalence therefore needs the explicit nonempty domain.

B-E09 Tree and forest operations on one example

Derivation and check

The unique simple path from a to f is a, b, d, e, f, of length 4. Deleting bd produces components $\{a, b, c\}$ and $\{d, e, f\}$. Adding cf creates the cycle c, b, d, e, f, c, of length 5.

Rooting at d gives the following values.

Sets, relations, functions, graphs, and trees solutions (continued)

Vertex	Parent	Depth	Height	Children	Graph degree
d	NIL	0	2	2	2
b	d	1	1	2	3
e	d	1	1	1	2
a	b	2	0	0	1
c	b	2	0	0	1
f	e	2	0	0	1

At the root, graph degree equals child count. Elsewhere, it exceeds the child count by 1. The degree sum is $10 = 2 \cdot 5$. Path uniqueness follows from the tree property, rather than from displaying just one route.

B-E10 Binary empty slots and counts

Derivation and check

Node	Left slot	Right slot
r	NIL	s
s	t	NIL
t	NIL	NIL

The height is 2, and there are four missing positions. The extended tree has $I = 3, L = 4$.

In general, a nonempty binary tree with n real nodes has $2n$ child slots. Exactly $n - 1$ slots hold real child edges, leaving $2n - (n - 1) = n + 1$ missing positions. The extended tree has $2n$ edges and $2n + 1$ vertices, consistent with the tree edge count.

A shared NIL sentinel reuses one implementation object. Logical positions are identified by the pair $(\text{parent}, \text{side})$ and remain distinct. Four positions do not require four allocations.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Sets, relations, functions, graphs, and trees solutions (continued)

B-E11 Full versus complete k-ary

Derivation and check

Every internal node emits k child edges. Since the tree has $I + L$ vertices,

$$kI = I + L - 1, \qquad L = (k-1)I + 1.$$

At $k = 3, I = 5$, this gives $L = 11$, a total of 16 vertices, and 15 edges.

If all leaves also had a common depth h, then

$$I = \frac{3^h - 1}{2}.$$

The value $I = 5$ would require $3^h = 11$, impossible for an integer $h \geq 0$. Fullness alone does permit $I = 5$. Start with a root and three leaves, then expand one leaf into three children four times. For $k = 1$, full trees are chains with $L = 1$; the complete-tree division formula needs a separate case.

B-E12 Bipartite graphs and odd cycles

Derivation and check

Every edge in a bipartite graph switches sides. An odd number of traversed edges cannot return to the starting side, so no odd cycle exists.

Conversely, suppose a finite simple undirected graph has no odd simple cycle. Root each component and color by the parity of shortest-path distance from the root. If an edge joins endpoints of equal parity, follow a root-to-u path, that edge, and a reversed root-to-v path. This is an odd closed walk.

At any repeated internal vertex, split the walk into two shorter closed walks. Their total length is odd, so retain an odd part. Repeating this reduction eventually leaves no internal repetition, hence an odd simple cycle, contrary to the premise. A simple undirected graph has no length-one loop, and a length-two return is even, so no odd degenerate case remains. Thus the coloring is proper and the graph is bipartite.

The quadrilateral a, b, c, d, a has sides $\{a, c\}$ and $\{b, d\}$. Adding the diagonal ac creates the triangle a, b, c, a and forces adjacent vertices a, c to have the same color. Removing ac restores the stated sides.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Sets, relations, functions, graphs, and trees solutions (continued)

Verification code appendix: sets_graphs_checks.py

```
"""Finite graph checks including exhaustive six-equivalence tests n<=5."""
from itertools import combinations
def components(n,edges):
 adj=[set() for _ in range(n)]
 for u,v in edges:adj[u].add(v);adj[v].add(u)
 seen=set();out=[]
 for root in range(n):
  if root in seen:continue
  todo=[root];seen.add(root);part=[]
  while todo:
   u=todo.pop();part.append(u)
   for v in adj[u]:
    if v not in seen:seen.add(v);todo.append(v)
  out.append(part)
 return out
def acyclic(n,edges):
 par=list(range(n))
 def find(x):
  while par[x]!=x:x=par[x]
  return x
 for a,b in edges:
  a=find(a);b=find(b)
  if a==b:return False
  par[a]=b
 return True
if __name__=='__main__':
 cases=0
 for n in range(1,6):
  possible=list(combinations(range(n),2))
  for mask in range(1<<len(possible)):
   ed={e for i,e in enumerate(possible) if
mask>>i&1};c=len(components(n,ed));forest=acyclic(n,ed);tree=forest and c==1
   assert (c==1 and len(ed)==n-1)==tree
   assert (forest and len(ed)==n-1)==tree
   assert (c==1 and all(len(components(n,ed-{e}))>1 for e in ed))==tree
   assert (forest and all(not acyclic(n,ed|{e}) for e in set(possible)-ed))==tree
   if forest:assert len(ed)==n-c
   assert sum(sum(u in e for e in ed) for u in range(n))==2*len(ed)
   cases+=1
 s=set(range(-3,6));classes={frozenset(y for y in s if (x-y)%4==0) for x in s}
 assert sum(len(c)**2 for c in classes)==21
 div={2,3,4,6};covers={(a,b) for a in div for b in div if a!=b and b%a==0 and not any(z not
in [a,b] and z%a==0 and b%z==0 for z in div)}
 assert covers=={(2,4),(2,6),(3,6)}
 print(f'PASS finite {cases} simple undirected graphs n1..5; forest edge identity, four
equivalent criteria, degrees; mod4 classes and covers')
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sets, relations, functions, graphs, and trees solutions (continued)

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study